

CAMPUSFLOW

Setup, user & test guide

A practical handover guide for a new machine, a guided demonstration, and role-by-role acceptance testing.

Prepared 16 September 2026 • Repository: NGA828/CampusFlow • Code baseline: e4062f7 (session branch arena/01a0a9b4-campusflow).

This guide describes the checked-out implementation, not a promise that every workflow is production-ready. The source review covered routes, controllers, migrations, seed data, web/mobile screens, API clients, scripts and tests. No application functionality was changed to prepare this document.

NEW MACHINE → FIRST LOGIN → ROLE WORKFLOWS → ACCEPTANCE

95 test scenarios • Web + mobile • Editable source included

Contents

1. Read this first	4
What you are installing	4
Important corrections to the quick start	4
What was actually checked for this guide	4
2. Set up on a new machine	5
2.1 Prerequisites and recommended environment	5
2.2 Obtain the code and install locked dependencies	5
2.3 Start a disposable local database	6
2.4 Configure Laravel before migrating	6
2.5 Configure and launch the web app	7
2.6 Launch mobile on a phone or emulator	7
2.7 Optional AI planner	8
2.8 First-run checklist and stopping	8
3. Accounts, data and a guided demonstration	9
Demo users	9
A 30-minute handover call	9
4. How to use the application	10
4.1 Public and common account actions	10
4.2 Student on web: planning and records	10
4.3 Student on mobile: physical workflows	10
4.4 Staff on web and mobile	11
4.5 Admin on web and mobile	11
5. Test environment and execution rules	13
5.1 Protect the database before automated tests	13
5.2 Commands and what they establish	13
5.3 Manual acceptance preparation	14
5.4 Rules applied to every action below	14
6. Action-by-action acceptance catalogue	16
6.1 Public, authentication and account	16
6.2 Student planning, discovery and engagement	17
6.3 Student mobile positioning and navigation	19

6.4 Room queues: student plus assigned staff	20
6.5 Office service lifecycle	22
6.6 Staff teaching, rooms and content	23
6.7 Admin people and governance	24
6.8 Admin campus and spatial configuration	25
6.9 Admin academics and service configuration	27
6.10 Cross-cutting release checks	28
7. Existing automation and API testing reference	29
Feature-suite map	29
API request conventions	29
Evidence and release decision	30
8. Known gaps and troubleshooting	31
Source-observed limitations at this baseline	31
Troubleshooting by symptom	31
9. Operational handover and production cautions	33
10. Source reference and maintenance	34

1. Read this first

What you are installing

CampusFlow has three supported runtime pieces: Laravel 13 API in backend/, Next.js 16 / React 19 web client in frontend/, and Expo SDK 57 mobile client in mobile/. PostgreSQL stores the authoritative application data. The server/ directory contains an alternative Fastify implementation; it is not started by the root launcher and is not the backend used in this guide. Do not start it alongside Laravel on the same port.

The web browser uses /api/v1 on its own origin; frontend/server.mjs proxies requests to Laravel on port 8001. A native phone calls the Laravel URL configured in EXPO_PUBLIC_API_URL. Web and mobile share accounts and data, but intentionally expose different actions.

Important corrections to the quick start

- npm run setup does not create backend/.env, generate APP_KEY, provision PostgreSQL, or install the root check dependency. Complete those steps first, or use the explicit installation below.
- backend/phpunit.xml currently names the ordinary campusflow database. Tests use RefreshDatabase and can destroy data. Isolate the test database before running npm test.
- The seed script creates campus entities with create(), not updateOrCreate(). Do not repeatedly run db:seed on a populated database. A fresh reset is destructive.
- PostgreSQL is required by the current migrations, including PostgreSQL-specific constraint/index SQL. Do not substitute SQLite for the test or development instructions.
- The health endpoint tolerates absent PostGIS. Installing PostGIS does not mean the schema has been converted to spatial geometry columns.
- Web reset-password currently submits fewer fields than Laravel requires. Treat that UI journey as a known issue, not a passing setup requirement.
- The web realtime client expects /api/ws, but the Laravel route/channel configuration does not implement that custom protocol. Starting Reverb alone is not a demonstrated fix. Verify changes by refresh; do not promise live push delivery.

What was actually checked for this guide

PASS: PHP static parser (130 files, zero syntax errors); API contract checker (199 routes, 185 web calls, 82 mobile calls, 127 test calls; zero problems or orphan routes); web and mobile TypeScript checks; mobile responsive helper tests (3/3).

NOT RUN: Laravel runtime/feature tests (PHP and Composer unavailable in this review environment); browser Playwright suite; production builds; physical-device, GPS, camera, push, concurrency, and end-to-end acceptance tests. The manual cases below are procedures to execute, not completed test results. Dependency installation reported 14 moderate vulnerabilities in the mobile dependency tree; triage with npm audit before release, rather than applying a forced upgrade blindly.

2. Set up on a new machine

2.1 Prerequisites and recommended environment

Use Git, Node.js 22 LTS with npm, Composer 2, PHP 8.3 or later (8.4 recommended), and PostgreSQL 17. The root manifest says Node ≥ 20 , but the locked Next/Expo dependencies can have stricter minimums: Node 22 is the safer baseline. Composer's locked dependencies are authoritative; do not bypass platform checks.

PHP needs the Laravel extensions (ctype, curl, dom/XML, fileinfo, filter, hash, mbstring, openssl, pcre, PDO, session, tokenizer) plus pdo_pgsql/pgsql for PostgreSQL. Install zip/unzip support for Composer. Use Composer's platform check to identify anything else missing.

On Windows, WSL2 Ubuntu is the simplest way to use the shell commands in this guide consistently. Install Node, PHP, Composer and Git inside WSL rather than mixing Windows and Linux binaries. Native Windows also works using PowerShell equivalents below; mobile LAN access from WSL may require port forwarding/firewall configuration. On macOS use your normal package manager; iOS native builds additionally require macOS and Xcode. Android native builds require Android Studio, Android SDK, emulator/device and the JDK required by the Expo build tools.

Choose either an existing PostgreSQL installation or Docker Desktop/Engine with Compose support. Docker is only used for the database here; there is no checked-in full-stack Docker deployment.

```
git --version
node --version
npm --version
php --version
php -m
composer --version
```

Check free ports 3000 (web), 8001 (API), 5432 (PostgreSQL) and 8081 (usual Expo Metro port). Have a browser, a terminal for the API/web launcher and a second terminal for mobile/tests.

2.2 Obtain the code and install locked dependencies

```
git clone https://github.com/NGA828/CampusFlow.git
cd CampusFlow
git rev-parse --short HEAD
npm ci
composer install --working-dir=backend
composer check-platform-reqs --working-dir=backend
npm --prefix frontend ci
npm --prefix mobile ci
```

For an exact reproduction, obtain the reviewed revision from your maintainer and verify the printed SHA; main may have moved since this guide. Authenticate through your normal GitHub connection if the repository is private. Never place credentials in a clone URL or in documentation.

npm ci uses the committed lockfiles; use it for repeatable setup rather than updating dependencies. The root npm ci installs php-parser used by the checks. Do not run composer update or npm audit fix --force as a setup step. backend/package.json belongs to Laravel's asset scaffold; the supported web UI is frontend/, not the Laravel Vite app.

2.3 Start a disposable local database

Option A: PostgreSQL/PostGIS in Docker (choose your own local-only password and substitute it consistently). The volume keeps data when the container stops. Publishing on 127.0.0.1 avoids exposing the database to the LAN.

```
docker run --name campusflow-db \
  -e POSTGRES_USER=campusflow \
  -e POSTGRES_PASSWORD=CHANGE_THIS_LOCAL_PASSWORD \
  -e POSTGRES_DB=campusflow \
  -p 127.0.0.1:5432:5432 \
  -v campusflow-pgdata:/var/lib/postgresql/data \
  -d postgis/postgis:17-3.5

docker exec campusflow-db pg_isready -U campusflow -d campusflow
docker exec campusflow-db psql -U campusflow -d campusflow \
  -c 'CREATE EXTENSION IF NOT EXISTS postgis;'
```

Wait for `pg_isready` to report accepting connections, then retry the extension command if necessary. For later sessions use `docker start campusflow-db`, not another `docker run`. If 5432 is occupied, map 5433:5432 and use `DB_PORT=5433`. Do not delete the volume unless you intend to erase every database in it.

Option B: in your locally installed PostgreSQL, have an administrator create a dedicated login and database. Run these in `psql` as the PostgreSQL administrator, substituting a password:

```
CREATE ROLE campusflow LOGIN PASSWORD 'CHANGE_THIS_LOCAL_PASSWORD';
CREATE DATABASE campusflow OWNER campusflow;
-- Connect to campusflow before running the next statement.
CREATE EXTENSION IF NOT EXISTS postgis;
```

PostGIS must be installed on the PostgreSQL server before enabling the extension. Extension creation may need elevated database privileges. If unavailable, core local flows may still run; `/health` reports `postgis: null`. Record that limitation for spatial QA.

2.4 Configure Laravel before migrating

```
cp backend/.env.example backend/.env
```

Edit `backend/.env` (never commit it). Keep the existing remaining defaults, but change these values to your machine:

```
APP_NAME=CampusFlow
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8001
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=campusflow
DB_USERNAME=campusflow
DB_PASSWORD=CHANGE_THIS_LOCAL_PASSWORD
SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database
BROADCAST_CONNECTION=log
MAIL_MAILER=log
OPENAI_API_KEY=

php backend/artisan key:generate
php backend/artisan config:clear
php backend/artisan migrate --seed
```

```
php backend/artisan migrate:status
```

Expected: a generated APP_KEY, all migrations recorded as run, and seeded records. Tables for sessions/cache/jobs are created by the migrations. The example Redis settings do not make Redis a prerequisite for these database-backed defaults. Log mail is for local testing, not email delivery.

If migrations fail, fix the connection/extension/permissions issue before retrying. If seeding partially completed, do not blindly seed again: inspect the disposable database or reset it intentionally. On a genuinely disposable local database only, `npm run db:reset` runs `migrate:fresh --seed` and erases existing tables and data. Take a backup before any reset of data you want to retain.

2.5 Configure and launch the web app

Create `frontend/.env.local` containing:

```
NEXT_PUBLIC_API_URL=/api/v1
```

With the standard ports, no other frontend variables are needed. Start from the repository root:

```
npm run dev
```

Open `http://localhost:3000` and `http://localhost:8001/api/v1/health`. Healthy means HTTP 200 with `success=true`, `data.status=ok` and `data.database.ok=true`. Check the same endpoint through `http://localhost:3000/api/v1/health` to verify the proxy as well as Laravel.

If the hostname inherited by your shell is not a bindable address, start with `HOSTNAME=0.0.0.0 npm run dev`. The custom web server reads environment variables, not the `--hostname/--port` flags a normal Next CLI would read.

For non-default ports set all relevant server-side variables in the launcher shell:

```
HOSTNAME=0.0.0.0 API_PORT=8002 WEB_DEV_PORT=3001 \
  API_PROXY_TARGET=http://127.0.0.1:8002 \
  API_INTERNAL_URL=http://127.0.0.1:8002 npm run dev
```

Changing `API_PORT` alone does not change the web proxy target. Keep `NEXT_PUBLIC_API_URL` relative: browser `localhost` always means the user's own machine, not a remote server. `API_PROXY_TARGET` and `API_INTERNAL_URL` are server-side addresses without `/api/v1`. The internal URL supports server-rendered reads. Restart after configuration changes.

PowerShell equivalents: use `Copy-Item` instead of `cp`, and `$env:NAME="value"` before running a command. For example:

```
Copy-Item backend/.env.example backend/.env
$env:HOSTNAME="0.0.0.0"
$env:API_PROXY_TARGET="http://127.0.0.1:8001"
npm run dev
```

Use one-line Docker commands in PowerShell, or PowerShell backticks rather than the shell backslashes shown above.

2.6 Launch mobile on a phone or emulator

Find your computer's LAN IPv4 address (for example `192.168.1.20`). Put the phone and computer on the same reachable network. Allow inbound TCP 8001 and the Expo development-server port on a trusted network; do not expose PostgreSQL.

Create `mobile/.env`:

```
EXPO_PUBLIC_API_URL=http://192.168.1.20:8001/api/v1
```

Replace the example address. Keep the /api/v1 suffix. Open that health URL in the phone browser first; if it cannot load, Expo cannot fix the network problem.

```
cd mobile
npm run start
```

Scan Metro's QR code with a compatible Expo Go app, or press a for Android / i for an iOS simulator. This QR code launches the app; it is not a campus positioning anchor. If the installed Expo Go does not support SDK 57, install a compatible version or use a development build. Android emulator can commonly reach the host at 10.0.2.2; physical devices cannot use host localhost. iOS simulator can use the Mac host address, but LAN configuration is more portable.

For a native development build with the platform toolchain installed:

```
cd mobile
npx expo run:android
# On macOS, for iOS instead: npx expo run:ios
npx expo start --dev-client
```

Camera/location permissions are required only for the associated physical actions; denial must not break login or ordinary reading. Push registration is intentionally skipped in Expo Go, browser targets and non-physical devices. A compatible physical-device development build plus Expo/EAS project configuration is needed to test token registration. Registration is not proof of backend push delivery.

2.7 Optional AI planner

With no key the deterministic planner is available for local tests. To use OpenAI, add a real key only to backend/.env, configure OPENAI_MODEL=gpt-4o-mini, OPENAI_BASE_URL=https://api.openai.com/v1 and OPENAI_TIMEOUT_SECONDS=20, then run php backend/artisan config:clear and restart the API. External usage may incur charges. Never copy the key into frontend or EXPO_PUBLIC variables.

GET /api/v1/ai/capabilities with an authenticated token reports the configured planner. Verify backend-authorized results rather than assuming model text equals a successful action. OpenAI tests use mocks; they do not establish live provider availability.

2.8 First-run checklist and stopping

- Both direct and proxied health requests succeed.
- Landing page displays real campus information rather than a loading/error state.
- Each seeded resident account signs in to the appropriate workspace.
- Student sees STB rooms and the seeded academic/service data.
- A phone can reach health and sign in independently.
- No secret files have been added to Git; git status --short is reviewed.

Stop the root launcher with Ctrl+C; it shuts down its child processes. Stop Metro separately. Stop the database with docker stop campusflow-db if used. Starting services again does not require re-seeding.

3. Accounts, data and a guided demonstration

Demo users

All three resident demo passwords are password123: student@campusflow.edu (Alex Rivera, CS-2026-042), staff@campusflow.edu (Dr. Jane Smith), admin@campusflow.edu (System Administrator). These are local demo credentials, never production credentials. A seeded visitor record also exists, but public browsing should be tested signed out; visitor tokens do not gain resident workspaces.

Seeded anchors include QR-STB-F1-MAIN and QR-STB-F1-LAB101. Campus examples include Science & Technology Building (STB), Student Union Building (SUB), rooms STB-101, STB-102 and STB-201, and offices REG and FIN. The term is 2026-FALL; courses include CS-101 and CS-305. Dates, weekday and term membership affect the timetable. Event times are relative to seeding time. Do not treat an empty today view on a non-class day as a broken installation.

The seed campus is around latitude 37.775 and longitude -122.419 in San Francisco. A tester on another continent is genuinely outside its proximity radius. For physical acceptance tests create a separate QA building/room/office with correct local coordinates and valid geofences/anchors, or use a controlled simulator location. Never disable production proximity rules merely to make a demo pass.

A 30-minute handover call

1. Minutes 0–5: show the three folders, health endpoint and proxy; explain web versus mobile; confirm database safety.
2. Minutes 5–10: browse signed out, then sign in as student. Open timetable, room detail, queue board and office services. Explain that the web is for planning, not physical room admission/check-in.
3. Minutes 10–18: use student mobile at a valid test location; scan an issued campus anchor, open a room and take a queue ticket. In a separate staff browser, open the assigned line, call, check in/admit as appropriate, then complete. Refresh both clients and compare the server-confirmed status.
4. Minutes 18–23: take an office ticket as student; use the staff office desk to call, verify/check in, start service when eligible, and complete. Show the student's history.
5. Minutes 23–28: use admin web to show users/scopes, campus, spatial data, academics, services and alerts. Demonstrate a disposable edit, then restore it.
6. Minutes 28–30: show checks, known limitations, evidence template and next actions. Do not present failed realtime or reset-password UI behavior as a working feature.

Use separate browser profiles/incognito sessions for student, staff and admin to avoid overwriting tokens. The same account across web and mobile shares backend state.

4. How to use the application

4.1 Public and common account actions

Start at / for the public campus overview and links offered there; /status shows service health. Public endpoint availability does not imply every endpoint has its own standalone public page. Use /register for self-service student registration; privileged staff/admin accounts must be created by an administrator. Registration does not allow the caller to choose a privileged role.

Use /login to sign in. The session routes you into the correct role workspace. /account provides shared account settings; students also have /student/profile. Edit profile fields, save and reload to confirm persistence. Change password with current password and matching confirmation. Use Sign out when changing roles on a shared machine. /forgot-password and /reset-password exist, but the current reset submission mismatch is documented in section 8.

Open Notifications (shared /notifications or student /student/notifications), apply available filters, open an item, mark one read or mark all read. Unread counts should change only after confirmation. Appearance and other device preferences, where offered, must be distinguished from server-saved profile fields; test persistence on that device rather than assuming cross-device sync.

4.2 Student on web: planning and records

/student/dashboard is the starting point for next class, activity and service summaries.

/student/timetable shows the personal schedule derived from enrolments. Move between weeks/days and open room links. /student/campus/map lets you inspect buildings/floors and search; /student/campus/rooms and /student/campus/rooms/[code] provide room details and availability. Web route preview is planning, not live positioning.

/student/services/queues shows the queue board, active ticket information and history/cancellation controls where applicable. Take a physical room-admission ticket from mobile, not web.

/student/services/offices lists offices; open /student/services/offices/[code], inspect availability and request an eligible ticket. Open the ticket detail to see number, position, estimates/history and cancellation. Proximity check-in is mobile-only even when an office ticket can be requested on web.

/student/campus/events shows events with registration/cancellation; /student/announcements shows notices. /student/assistant supports prompts, conversation history/new conversations and deletion. Ask for campus facts or your schedule and follow authorized links. Do not assume a suggested action has executed unless the server returns confirmed data.

4.3 Student on mobile: physical workflows

The student tab shell provides Home, Map, Queue, Scan and More. More opens timetable, offices, assistant, notifications and profile. Native routes are under mobile/src/app/student/, not the old (tabs) paths listed in some older documents.

For positioning, open Scan, grant camera permission and scan an admin-issued campus QR payload; use the manual fallback if offered. The backend validates the anchor. A scan may establish an indoor position without GPS; invalid, revoked or stale payloads must give a rejection rather than silently selecting a room.

For navigation, choose a room and Navigate. Select the available origin/accessibility preference, start a session and follow route steps. Permit location access for live updates. Test off-route/recalculation and end the session explicitly. An accessible route requires correctly configured graph edges; the UI

preference cannot create a missing step-free path.

For room admission, open a room/queue, confirm it is open and join with the required location/scan evidence. Observe the ticket position and ETA. When called, use navigating/check-in actions if offered and physically approach the room; staff manage admission/completion. Cancel an unneeded ticket or use leave when available. Only enabled, server-authorized transitions should be offered.

For offices, open More > Offices, choose a desk/service and request a ticket if eligible. Follow the estimated service window, mark approaching if offered, and check in at the office using location. In the current controller, student office check-in sets `in_service` immediately, rather than a separate `checked_in` status. Do not start service a second time on an already `in_service` ticket.

4.4 Staff on web and mobile

`/staff/dashboard` summarizes assigned work. `/staff/queues` lists assigned lines, including closed lines; open `/staff/queues/[id]` to inspect the line, open/close admission, call next or a selected ticket, verify/check in, admit, complete or mark no-show. These are different state transitions, not interchangeable shortcuts. Keep another student session open to verify the corresponding view.

`/staff/offices` and `/staff/offices/[id]` provide desk operations: call next/selected, student registration-number lookup, check-in, start service, complete and no-show. Work only within the assigned scope. Assignments and office staff links are controlled by admin.

`/staff/rooms` allows scoped room-status correction, not infrastructure/capacity editing. `/staff/timetable` supports create, edit and delete for authorized teaching entries. `/staff/content` supports event creation/edit/deletion and announcement creation/deletion; do not assume announcement edit is available because event edit is. Use the actual published/audience fields provided by the forms.

Staff mobile offers quick queue and office operations, with line/office detail and More. It is not the timetable/content-authoring console. Use web for those tasks. Attempting student scanning or student ticket-taking with a staff token is not a supported shortcut.

4.5 Admin on web and mobile

`/admin/dashboard` provides management summaries. `/admin/users` manages people, role changes, password resets and staff assignments; the permission registry is a read view, not an arbitrary permission editor. Role changes are separate from ordinary profile edits. Protect the last administrator and do not demote the current administrator accidentally.

`/admin/campus` manages buildings, floors and rooms. Create parents before children; select a building/floor to limit the working scope. Edit plan dimensions/room geometry using the fields exposed in the interface; test floor-plan/spatial actions in `/admin/spatial` as applicable. Do not assume a general file-upload or freehand floor-plan editor exists merely because a floor plan is displayed.

`/admin/spatial` manages QR anchors/payloads and regeneration, navigation nodes/edges and geofences. Configure the graph before expecting navigation. Reissue printed QR codes after regeneration.

`/admin/academics` manages terms, courses, enrolments and the timetable overview. Timetable entry operations use the authorized staff authoring contract where exposed.

`/admin/services` configures room queues, offices, service windows and office staff. Set admission policy, capacities, radii, operating hours and assignments deliberately. This is distinct from staff operating a live ticket line.

`/admin/analytics` shows metrics and audit-log views where exposed. `/admin/alerts` shows derived operational conditions; acknowledge a condition with an optional note. Acknowledging does not repair

the underlying condition. /admin/settings edits named supported settings one at a time.

Admin mobile is monitoring and alerts, including alert detail and acknowledgement. It intentionally does not provide full user/campus/service configuration. Switch to admin web for those operations.

5. Test environment and execution rules

5.1 Protect the database before automated tests

Never run the current default npm test against data you want to keep. backend/phpunit.xml points to campusflow; RefreshDatabase can rebuild its schema. Create a separate campusflow_test database and a test-only PostgreSQL login that has no access to your normal database. The login should own the test database/schema so migrations can create and drop tables. Provision PostGIS there separately if required.

For the Docker example, connect as the provisioning administrator and create the isolated login/database:

```
CREATE ROLE campusflow_tester LOGIN PASSWORD 'CHANGE_THIS_TEST_PASSWORD';
CREATE DATABASE campusflow_test OWNER campusflow_tester;
REVOKE CONNECT ON DATABASE campusflow FROM PUBLIC;
GRANT CONNECT ON DATABASE campusflow TO campusflow;
```

Apply access changes only on this disposable local installation; coordinate database access rules with your DBA elsewhere. Connect as campusflow_tester to verify it can access campusflow_test and cannot access campusflow. Do not run tests as the broad Docker POSTGRES_USER account.

Copy backend/.env to backend/.env.testing, give it a separate APP_KEY and test-only credentials, set APP_ENV=testing, DB_DATABASE=campusflow_test, and use array cache/session plus sync queue. Set OPENAI_API_KEY empty. Generate its key from backend/ using php artisan key:generate --env=testing. Also change every DB_* entry in your LOCAL backend/phpunit.xml to the same test database, host, port and test-only credentials, because its explicit defaults can defeat an .env.testing-only setup. Do not commit passwords in phpunit.xml; keep a sanitized template if you formalize this configuration.

Before each suite, from backend/ run php artisan config:clear, then php artisan config:show database --env=testing and inspect the effective connection locally (do not share password output). Review phpunit.xml again; an artisan environment display alone does not prove PHPUnit's overrides are safe. If in doubt, stop. Never rely only on the name APP_ENV=testing to prevent data loss.

5.2 Commands and what they establish

```
# From repository root: static gates
npm run check
npm run lint

# Only after test-database isolation above
npm test
php backend/artisan test --filter=QueueTest
php backend/artisan test --filter=PlatformRoleAccessTest

# Mobile helper tests and packaging check
npm --prefix mobile run test:responsive
cd mobile
npx expo export --platform web
cd ..

# Web packaging check
npm --prefix frontend run build
```

npm run check covers PHP syntax, API/platform contracts and both TypeScript clients; it does not run Laravel, lint or browser tests. A green route checker validates route existence/platform wiring, not payload completeness or business behavior. The password-reset mismatch is an example of that distinction.

For Playwright, keep the web server already running. Its config has no automatic webServer startup:

```
cd frontend
npx playwright install chromium
npm run test:responsive
```

Use RESPONSIVE_BASE_URL to target a different web origin. Several tests intercept API traffic with fixtures: they are useful for layout/state tests, not proof of real Laravel integration. Inspect each spec before interpreting a pass. Native mobile camera, GPS and push cannot be proven by browser fixtures.

Some specs need a separate mobile web export at port 3101 (check their environment variables). A same-origin mobile browser preview can be prepared with:

```
cd mobile
EXPO_PUBLIC_API_URL=/api/v1 npx expo export --platform web
npm run preview:web
```

Use a separate terminal for the tests. The mobile preview script proxies API requests; native builds must instead keep a reachable absolute API URL. Do not confuse the browser export with physical-device acceptance.

5.3 Manual acceptance preparation

Create disposable users QA Student A, QA Student B, QA Assigned Staff and QA Unassigned Staff through admin web. Create a QA campus area near your actual test location, a finite-capacity queue, one open office with a service window, a valid QR anchor and a connected navigation graph. Assign only the intended operator. Create a current term/course/enrolment, a published event and a targeted announcement. Record the actual IDs returned by the backend, rather than inventing UUIDs in API requests.

For each test record: ID, date, build SHA, platform/device/OS, role/account, preconditions, exact inputs, steps, expected versus actual result, network status/error code, screenshots/log references, Pass/Fail/Blocked/Not run and defect ID. Redact passwords, bearer tokens, reset tokens and personal data. Default every case to Not run; a known code issue is a predicted failure until reproduced.

Run destructive cases on QA-only data and clean up children before parents. Restore schedules/roles/settings after testing. Run multi-user cases in separate sessions. Refresh or re-login to verify durable changes rather than accepting a toast as evidence.

5.4 Rules applied to every action below

For every read, test normal data, empty data, loading, retry after network failure, search/filter reset and pagination where exposed. Expect clear empty/error states, no stale private data and no console crash.

For every form, test valid submit, required fields empty, invalid numeric/date bounds, duplicate identifiers, cancel without saving, repeated click, keyboard submit, backend rejection and reload persistence. A failed write must not show success or leave an invented row.

For every mutation, replay with no token, the wrong role, the wrong platform and another user's/out-of-scope resource. Expect 401 unauthenticated, 403 forbidden (or deliberate 404 to hide a resource), and 422/409 for invalid business requests as implemented. Record the specific code; do not require all errors to be 422. Changing X-CampusFlow-Client never grants a role permission.

For every delete, first cancel its confirmation, then confirm on an unused QA object and verify removal after reload. Test linked/in-use objects separately: reject safely or perform only the documented cascade, never silently orphan dependent data. These shared checks apply to every individual create/edit/delete action in the catalogue, not just one sample per screen.

6. Action-by-action acceptance catalogue

Each case combines an action, executable steps and an expected result. Run the universal checks in section 5.4 for every action named. The feature-file references in section 7 point to existing automation, not complete coverage of all cases.

6.1 Public, authentication and account

AUTH-01 — Public browse and service status

Steps: Signed out, load /, follow its campus links and open /status; request /public/overview and /health.

Expected: Public data only; health agrees with database reachability; no student identity/tickets exposed.

AUTH-02 — Student registration

Steps: Register a unique QA email with matching password confirmation; repeat email, short password and mismatched confirmation; attempt role=admin in an API request.

Expected: Valid account is a student; invalid inputs rejected; caller cannot self-register as staff/admin.

AUTH-03 — Sign in and session restoration

Steps: Sign in with each role, refresh, close/reopen app; repeat with incorrect password and a disabled QA user.

Expected: Correct role workspace and restored authorized session; bad credentials/inactive user denied.

AUTH-04 — Sign out and expired token

Steps: Sign out, use browser back, then request /me with the revoked token; repeat using an expired/invalid token.

Expected: Private requests fail; client returns to sign in and removes stale private content.

AUTH-05 — Profile editing

Steps: Open account/profile, edit name/phone/department/avatar only where offered; cancel another edit; refresh.

Expected: Allowed fields persist; cancelled edit does not; client cannot change role through profile payload.

AUTH-06 — Change password

Steps: Submit wrong current password, then a valid new password with confirmation; log out and sign in again.

Expected: Wrong current password fails; new password succeeds; old password no longer authenticates. Restore QA password afterward.

AUTH-07 — Request password reset

Steps: Submit known and unknown email via forgot-password; inspect local mail log or development response privately.

Expected: No crash; request has a meaningful result; development token is never exposed when APP_DEBUG=false. Log delivery is not actual email delivery.

AUTH-08 — Complete password reset

Steps: Follow reset UI with a fresh token; inspect its payload. Separately test API with email, token, password and password_confirmation; replay token.

Expected: UI is predicted to fail validation because required fields are omitted. Correct API flow should reset once and revoke sessions; invalid/reused token rejected.

AUTH-09 — Notification inbox

Steps: Open notifications, use filters and paging, open an item, mark one read, then mark all; reload another signed-in view.

Expected: Own notifications only; count changes after server confirmation and stays consistent after reload; repeated read is safe.

AUTH-10 — Appearance/preferences

Steps: Change each appearance/notification preference actually offered; navigate away and restart app; compare a second device.

Expected: UI remains usable; persistence matches whether preference is local or server-backed; unsupported cross-device sync is not claimed.

AUTH-11 — Device registration

Steps: On a supported physical development build, allow notification permission and inspect POST /me/devices; repeat deny and Expo Go.

Expected: Token belongs to current user; repeat registration is safe; denied/unavailable push does not block app. Delivery is a separate test.

AUTH-12 — Role deep links

Steps: Open student URLs as staff/admin and admin URLs as student, on both clients; retry API directly with forged role in body.

Expected: UI redirects/refuses and backend independently rejects; no sensitive flash or role escalation.

6.2 Student planning, discovery and engagement

STU-01 — Dashboard and next class

Steps: Sign in as enrolled student; compare next class, today's sessions and active-ticket summaries with underlying detail views; use unenrolled student too.

Expected: Summary matches real enrolled data; sensible empty state without enrolment; no fabricated class.

STU-02 — Timetable controls

Steps: Select previous/next/current week and day; open class/room links; change test enrolment as admin and refresh.

Expected: Dates and room links are correct; only student's enrolled entries appear; removed enrolment disappears.

STU-03 — Campus map and building/floor selection

Steps: Search STB and QA building, select building then floor, change floors, clear selection, open a room.

Expected: Selected parent scopes rooms/plan correctly; missing plan/coordinates are handled without crash.

STU-04 — Room directory and filters

Steps: Search by room code/name, combine offered filters, clear them, page results and try no-match text.

Expected: Results match all active filters, including parent scope; reset restores full list; no-match state is clear.

STU-05 — Room detail and availability

Steps: Open STB-101 and QA room, inspect capacity/features/status, inspect schedule/free slots, follow queue/map links.

Expected: Fields match backend; availability changes with timetable; unknown room shows not-found rather than another room.

STU-06 — Route preview on web

Steps: Use offered origin/destination and accessibility controls; compute preview; request disconnected/invalid endpoints.

Expected: Connected route is coherent; no live scanning/tracking UI; unavailable route is explicit, not a straight-line invention.

STU-07 — Queue board and own history

Steps: View open/closed queues, select/filter rows, inspect ticket details/history and active summary; repeat with another student.

Expected: Queue state matches staff view; no web join/check-in action; ticket ownership enforced.

STU-08 — Event discovery and registration

Steps: Filter/open a published QA event, register, repeat registration, cancel registration and reload; test full/closed event.

Expected: Registration/cancellation persists; capacity and duplicate rules hold; full/closed enrollment is rejected as appropriate.

STU-09 — Announcements and targeting

Steps: Browse/read notices with each role and a second department; test future/draft/targeted notices through public and resident APIs.

Expected: Only intended published content visible; audience filtering is server-side, independent of client platform.

STU-10 — Office directory and details

Steps: Search/select REG or QA office, inspect status, hours, service windows and location; follow detail link.

Expected: Data agrees with admin settings; closed/unavailable state is clear; invalid code handled safely.

STU-11 — Office ticket request on web

Steps: Request an eligible ticket, inspect detail/number/position/history, double-click and refresh; test closed office and configured daily limit.

Expected: At most one applicable active ticket; correct office; constraints enforced; check-in remains mobile-only.

STU-12 — Student assistant prompts and history

Steps: Ask for next class, room location and office information; start another conversation, reopen prior one, delete it and reload.

Expected: Answers/tools are role-authorized and grounded in backend; conversations are owned and deletion persists.

STU-13 — Assistant refusal and resilience

Steps: Ask to promote yourself, inspect another student's data, or execute SQL; simulate provider timeout and blank/oversized prompt.

Expected: No privileged tool or database bypass; clear validation/error/fallback behavior; no false success on an action.

6.3 Student mobile positioning and navigation

NAV-01 — QR camera and manual input

Steps: Grant camera, scan current admin-issued payload for QA anchor; repeat via manual input; navigate away/back.

Expected: Correct anchor/room/floor is returned; scan is not processed repeatedly; camera releases appropriately.

NAV-02 — Invalid/revoked/stale QR

Steps: Scan malformed payload, unknown code, revoked anchor, payload issued before regeneration and another room's QR at queue join.

Expected: Invalid evidence is rejected with actionable message; no incorrect position/ticket is created. Record any legacy code fallback separately.

NAV-03 — Permission denial/recovery

Steps: Deny camera/location, retry, later enable in OS settings; background/foreground app.

Expected: Read-only app still works; physical action explains missing permission; recovery works without reinstall.

NAV-04 — Start live navigation

Steps: From home/timetable/room, navigate to connected QA destination and grant location; inspect origin/floor/steps.

Expected: Server-backed navigation session and sensible steps; wrong-role and web session start rejected.

NAV-05 — Accessible and inter-floor routing

Steps: Route across floors normally, then with accessible preference; test graph with stairs-only path.

Expected: Accessible route excludes non-accessible edges or reports none; does not claim step-free access over stairs.

NAV-06 — Position update and off-route recovery

Steps: Move along controlled test route, deviate, remain off route through the displayed grace period, then return/recalculate.

Expected: Position/session update is visible; off-route/recalculation follows actual controller behavior; no endless recalculation or false arrival.

NAV-07 — End navigation and interrupted session

Steps: End session, attempt another update; separately disconnect network/background app and reconnect.

Expected: Ended session is not silently kept active; error/recovery is clear; stale coordinates are not presented as a fresh fix.

NAV-08 — Native layout and controls

Steps: Use map/search/room details on a small phone, rotate, increase text size and open keyboard; try back/deep links.

Expected: Essential actions remain reachable; content avoids safe-area/keyboard overlap; no student routes in staff/admin shells.

6.4 Room queues: student plus assigned staff

QUE-01 — Join eligible queue

Steps: As mobile student near configured QA room (or with accepted QR evidence), join open queue and reload ticket.

Expected: One waiting ticket, server position and estimate, correct queue/user; record request/response IDs.

QUE-02 — Proximity boundaries

Steps: Join/check proximity from inside, outside and at configured radius/polygon boundary; omit fix; test room without coordinates/geofence.

Expected: Server honors configured evidence policy; invalid/missing evidence rejected where required, never assumed near.

QUE-03 — Closed/full/unlimited queue

Steps: Attempt join when closed, at finite capacity, and with configured unlimited capacity; reopen and retry.

Expected: Closed/full join rejected; unlimited setting is not interpreted as zero capacity; eligibility shown accurately.

QUE-04 — Duplicate/retry join

Steps: Double-tap join and replay same Idempotency-Key; then use a different key for same active user; test another queue.

Expected: One applicable active ticket per user/queue; replay is safe; document additional duplicate-policy scope rather than assume global uniqueness.

QUE-05 — View position and history

Steps: Join A then B, call/complete A as staff and refresh B; inspect active ticket and history from web/mobile.

Expected: Ordering, status/history and displayed ETA agree with backend; no absolute realtime guarantee is inferred.

QUE-06 — Navigating and student check-in

Steps: Call ticket as staff; select navigating on mobile, approach room, check in with allowed evidence.

Expected: Called -> navigating -> checked_in where accepted; browser check-in denied; invalid terminal-state check-in rejected.

QUE-07 — Cancel/leave ticket

Steps: Cancel waiting ticket; separately leave an eligible occupied ticket; reload board and try repeated/terminal cancellation.

Expected: Ticket is no longer active; occupancy/position effects are correct; invalid repeat transitions are safe. Report state corruption as a defect.

QUE-08 — Open/close line as staff

Steps: Open assigned queue detail, toggle closed/open, refresh student directory and attempt joins in each state.

Expected: Confirmed open state persists; closing blocks new joins without deleting existing tickets.

QUE-09 — Call next and selected ticket

Steps: With two waiting QA tickets call next; repeat on empty line; separately call selected eligible ticket.

Expected: Correct scoped ticket changes to called; empty state safe; wrong/terminal ticket not silently recalled.

QUE-10 — Staff check-in and admit

Steps: Verify student, staff-check-in eligible ticket, admit it and inspect room occupancy; try admitting above capacity.

Expected: Valid transitions reflected on both clients; occupancy is correct; capacity cannot be bypassed by direct API.

QUE-11 — Complete and no-show

Steps: Complete admitted ticket; separately mark a called absent ticket no-show; repeat both actions.

Expected: Terminal status/history and capacity correct; duplicate/illegal transitions do not corrupt counts.

QUE-12 — Scope and ownership

Steps: Attempt every queue operation as unassigned staff, student and admin mobile; ask student B to read/cancel A's ticket.

Expected: Only intended roles/platforms/scopes succeed; ownership enforced at API, not just hidden buttons.

QUE-13 — Concurrent joins and last place

Steps: Using separate tokens and simultaneous HTTP requests, have 5 students compete for 1 remaining slot; repeat on empty queue.

Expected: No over-capacity admission, duplicate active position or multiple active tickets; rejected requests are handled, not raw SQL failures.

QUE-14 — Concurrent operators and timeouts

Steps: Have two assigned staff call-next/admit simultaneously; age a ticket beyond configured timeout and observe with/without reads.

Expected: Each ticket served at most once, ordering/capacity remain valid. Record timeout behavior; no scheduler-based expiration is assumed in this checkout.

6.5 Office service lifecycle

OFF-01 — Request, numbering and scheduling

Steps: Request QA office ticket for A then B; inspect number, position, wait/window; repeat at a second office.

Expected: Ticket belongs to selected office; sequence and estimates are coherent; limits/hours/windows enforced where configured.

OFF-02 — Remote/closed/limit rules

Steps: Attempt remote request with remote tickets disabled, closed office, exhausted daily limit and missing service availability.

Expected: Business rejection is explicit; no ticket silently issued; office setting changes affect eligibility after refresh.

OFF-03 — Approaching and mobile check-in

Steps: Mark waiting ticket approaching; check in inside radius, outside radius and without evidence.

Expected: Valid current implementation changes to in_service at student check-in; invalid proximity/platform denied.

OFF-04 — Cancel and history

Steps: Cancel waiting/approaching ticket, view history; attempt cancelling completed/no_show ticket and another student's ticket.

Expected: Correct status/history, released active slot; terminal or non-owned cancel rejected.

OFF-05 — Staff desk call and verification

Steps: As assigned staff open desk, call next/selected ticket, look up CS-2026-042 (or QA registration number); test unknown registration.

Expected: Correct ticket is called; scoped identity/outstanding work only; unknown student safely handled.

OFF-06 — Staff check-in/start/complete

Steps: Use fresh eligible ticket, verify/check in as staff, start service only if offered for current state, then complete.

Expected: State machine accepts valid path and rejects repeated/out-of-order actions; completed ticket leaves active line.

OFF-07 — No-show and empty desk

Steps: Mark eligible absent ticket no-show, then call next on empty desk; repeat terminal transition.

Expected: No-show is recorded; empty operation safe; no phantom active ticket or extra service start.

OFF-08 — Office assignment isolation

Steps: Try every desk operation as unassigned staff or staff assigned only elsewhere; change assignment and re-test.

Expected: Scope is enforced on reads and writes; assigning one office does not open unrelated desks.

OFF-09 — Office concurrency/idempotency

Steps: Simultaneously request twice for same student, request for several students and call next from two operators.

Expected: One active user/office ticket; unique/coherent numbering; each ticket acted on once; database errors do not leak.

6.6 Staff teaching, rooms and content

STA-01 — Operational dashboard and directories

Steps: Compare staff counts with assigned queues/offices; test staff with no scope; filter open and closed lines.

Expected: Only assigned work counted/listed; closed assigned lines are accessible; unassigned empty state is honest.

STA-02 — Room status correction

Steps: Edit status of assigned QA room in /staff/rooms, reload student detail; try changing capacity/geometry and unassigned room.

Expected: Only authorized status fields change; admin infrastructure fields remain protected.

STA-03 — Create timetable entry

Steps: Create valid QA course/room/term entry including a Sunday case; verify enrolled student's week view.

Expected: Stored day/time/room/course agree; irrelevant student sees none; invalid references/time ordering rejected.

STA-04 — Edit/delete timetable entry

Steps: Change QA entry time/room then refresh student; cancel deletion first, then confirm; try entry outside assignment.

Expected: Changes persist, deleted session disappears; scope protected; linked room availability updates.

STA-05 — Create/edit/delete event

Steps: Create QA event with room, date range, category, capacity and publication state; edit; then delete disposable event.

Expected: All exposed fields round-trip; published event reaches correct viewers; draft/invalid dates handled safely.

STA-06 — Create/delete announcement

Steps: Publish QA notice with priority/audience/time fields; compare roles; cancel then confirm deletion.

Expected: Targeting and dates preserved; only authorized author/scope can remove. No unsupported announcement-edit action assumed.

STA-07 — Content/teaching platform ceiling

Steps: Attempt authoring from staff mobile or direct mobile-header API; attempt modifying another author's content.

Expected: Web-only capability/scope enforced; mobile retains legitimate quick desk/queue actions.

6.7 Admin people and governance**ADM-01 — Dashboard, analytics and audit**

Steps: Compare dashboard totals/analytics with seeded plus QA data; filter/page audit entries after a known admin edit.

Expected: Consistent live counts; traceable audit record where implemented; no invented metrics; no student access.

ADM-02 — Search/filter/page users

Steps: Search name/email, switch role/status filters and pages, clear filters; select a record.

Expected: Results and selected identity remain consistent; filter reset and zero results safe.

ADM-03 — Create/edit/disable/delete user

Steps: Create disposable staff/student, edit ordinary fields, disable and test login, then delete eligible QA user.

Expected: Valid persistence; duplicate email invalid; inactive user denied; destructive action preserves dependent-data integrity.

ADM-04 — Change role and reset password

Steps: Change a QA student's role through the dedicated control; reset that user's password, sign in with returned credential privately.

Expected: Role changes are separate from profile; new privileges reflect backend; reset credential works and is not logged publicly.

ADM-05 — Administrator guardrails

Steps: Try self-demotion and demotion/removal of last administrator; try role field in ordinary profile update.

Expected: Safety protections prevent lockout or privilege bypass; no partial update after rejection.

ADM-06 — Role registry and assignments

Steps: Read registry, add scoped QA staff assignment with offered capabilities, verify staff reach, remove it and verify again.

Expected: Registry is read-only; scoped assignment changes actual access; invalid scope IDs/types rejected.

ADM-07 — Settings

Steps: Edit one supported setting, reload; submit wrong type/bounds/unknown key; cancel unsaved change.

Expected: Named validated setting persists only on success; unsupported arbitrary keys cannot silently become configuration.

ADM-08 — Alerts and acknowledgement

Steps: Trigger QA condition, open alert detail, acknowledge with note and repeat; alter underlying condition and refresh.

Expected: Acknowledgement is idempotent for fingerprint, note validated; changed condition gets appropriate fresh fingerprint; acknowledgement is not a repair.

ADM-09 — Admin mobile monitoring

Steps: Open monitoring, alerts and alert detail, refresh and acknowledge; attempt user/campus/settings deep links and APIs.

Expected: Monitoring/actions work as allowed; full configuration refused with platform ceiling; counts agree with web after refresh.

6.8 Admin campus and spatial configuration

CAMP-01 — Buildings create/edit/delete

Steps: Create QA building with code/name/status and coordinates, edit with zero and null coordinates, search it; delete only after children removed.

Expected: Valid zero distinct from missing location; duplicate/out-of-range values rejected; linked parent removal safely blocked.

CAMP-02 — Floors create/edit/delete

Steps: Under QA building create floor code/level/plan dimensions, including ground level zero; edit then delete unused floor; attempt changing parent.

Expected: Correct parent scope and metre dimensions persist; zero level searchable; immutable parent enforced.

CAMP-03 — Rooms create/edit/delete

Steps: Create QA room under floor with type/capacity/status/features/admission flags, edit each offered field; delete unused copy.

Expected: Room fields round-trip; linked-room deletion safe; invalid capacity and references rejected.

CAMP-04 — Room geometry and floor-plan fields

Steps: Edit exposed x/y/geometry/plan fields, change only one coordinate and reload; submit unsupported rectangle fields directly.

Expected: Supported partial update preserves unrelated metadata; unsupported fields rejected or explicitly ignored, never silently erase geometry.

CAMP-05 — Navigation nodes CRUD

Steps: Create two valid QA nodes tied to correct building/floor/room, edit coordinates/type, list/filter, delete unused node.

Expected: Nodes appear in correct scope; invalid references rejected; linked deletion does not leave broken graph.

CAMP-06 — Navigation edges CRUD

Steps: Connect QA nodes with weight/distance/accessibility/direction fields offered, edit and compute route, delete edge and recompute.

Expected: Routing honors stored edge data/direction; removing only connection yields explicit no-route; invalid/self links tested.

CAMP-07 — Geofences CRUD

Steps: Create radius and polygon fences where supported, edit boundaries, save and verify student proximity, delete disposable fence.

Expected: Geometry persists and is honored server-side; invalid radius/polygon rejected; fallback behavior after deletion documented.

CAMP-08 — QR nodes create/edit/revoke/delete

Steps: Create QA anchor linked to graph/floor, fetch displayed payload, scan on student mobile, edit/revoke/delete unused copy.

Expected: Issued anchor resolves correctly; revoked/deleted evidence refused; permissions restrict writes to admin web.

CAMP-09 — QR regeneration/export affordances

Steps: Capture old payload, regenerate, obtain new payload; use offered copy/download/print controls and scan both versions.

Expected: New output matches current version; old signed payload invalidated; export controls produce usable content, not empty files.

CAMP-10 — Spatial editor read/interaction states

Steps: Select buildings/floors/layers/tabs, search objects, pan/zoom and use each offered editor control; save/cancel pending edits.

Expected: Selection stable, unsaved state obvious, cancel restores prior data; no map crash on missing imagery/plan.

6.9 Admin academics and service configuration

CFG-01 — Terms CRUD

Steps: Create QA term with valid dates, edit dates/current flag where offered, view timetable, delete unused term.

Expected: Date order validated, current/selected term coherent; linked deletion safe.

CFG-02 — Courses CRUD

Steps: Create code/title/department QA course, edit fields and offered filters, delete unused course.

Expected: Unique code and required fields enforced; dependent enrolments/timetable not silently orphaned.

CFG-03 — Enrolment add/remove

Steps: Add QA student/course/term enrolment, attempt duplicate, view student schedule; remove enrolment and refresh.

Expected: Student gains/loses only matching timetable; invalid user/course/term rejected; duplicate safe.

CFG-04 — Academic timetable overview

Steps: Select term/course and inspect entries, use available authoring links as authorized; compare with staff and student.

Expected: Admin overview complete for scope; no unrelated personal student dashboard used as substitute.

CFG-05 — Queue create/configure/update/delete

Steps: Configure queue for a QA room without one, edit capacity/unlimited/open/proximity/duplicate/timeout fields offered, delete unused queue.

Expected: One configuration per room; limits validated and used by join/admit; active/in-use removal safe.

CFG-06 — Offices CRUD and scheduling

Steps: Create QA office linked to room, edit prefix/duration/hours/open/remote/limit fields, read student details and request ticket; delete unused copy.

Expected: All exposed settings persist and enforce eligibility; invalid hours/duration rejected; live tickets protected.

CFG-07 — Service windows CRUD

Steps: Create window for QA office with valid schedule/capacity fields, edit/close it, inspect student estimates; delete unused window.

Expected: Window stays scoped to office; impossible intervals/invalid references rejected; estimates reflect supported scheduling rules.

CFG-08 — Office staff add/remove

Steps: Add QA staff to one office with role/primary fields, inspect staff directory, remove and retry operations.

Expected: Assignments round-trip and affect access; duplicate/invalid member handled safely; unrelated office remains forbidden.

6.10 Cross-cutting release checks

SYS-01 — Offline, reconnect and failed writes

Steps: Disconnect network during each key read/write, reconnect and retry with same intent/key; stop API to provoke 502.

Expected: No false success; input retained where appropriate; no duplicate ticket on retry; recovery action visible.

SYS-02 — Realtime and notification delivery

Steps: Operate ticket in second session; observe without refreshing, inspect /api/ws upgrade, then refresh; separately test push on physical device.

Expected: Record actual delivery path and latency. Custom socket/push integration is not demonstrated; refreshed persisted state must still match.

SYS-03 — Accessibility and responsive coverage

Steps: Keyboard-only web navigation, visible focus, form labels/errors, screen reader, 200% zoom; narrow/wide desktop and native large text.

Expected: Critical actions readable/reachable; no horizontal clipping or color-only status; dialogs trap/return focus correctly.

SYS-04 — Privacy and abuse boundaries

Steps: Replay representative reads/writes without token, wrong role/platform/ownership; inspect public projections and assistant outputs.

Expected: No private identities/tickets/role escalation; app handles auth failure safely; capture missing throttling as hardening issue.

SYS-05 — Production-mode smoke and backup restore

Steps: Build both clients, serve web with production API/proxy configuration, test login/core flows; restore sanitized DB backup into separate DB.

Expected: Build/runtime both verified; no debug token exposure; restore usable without touching original data; deployment gaps documented.

7. Existing automation and API testing reference

Feature-suite map

- AuthTest: registration/login/profile/logout. Add/reset/negative cases from AUTH catalogue rather than assuming they are covered.
- MeTest and EngagementTest: own notifications and engagement reads/event registration.
- AcademicTest: personal timetable and course listing.
- CampusTest and CampusConfigurationTest: campus reads, parent scoping, coordinates, plan dimensions, geometry and safe deletion contracts.
- NavigationTest: QR scan and route computation; not a physical-device walk.
- QueueTest: listing/join/read/cancel/closed-line/ownership contract; not a proven concurrent stress test.
- OfficeTest: office list, ticket creation and owner read contract; not exhaustive service lifecycle coverage.
- StaffTest: scoped directories/dashboard, room status, teaching and content contracts.
- AdminTest: dashboard/users/buildings, people/role separation, assignments and alert fingerprint/acknowledgement.
- PlatformRoleAccessTest: role/platform ceilings, ownership, public-data separation and announcement targeting. Read assertions, not only test names: some names retain wording from older implementations.
- AiAssistantTest: deterministic planner and mocked OpenAI tool selection.
- HealthTest, PublicOverviewTest and ExampleTest: basic health/overview/framework checks.
- frontend/tests/responsive/: layout, alerts, coordination, campus admin/operations, staff services, student companion/discovery/services and mobile/web visual specs, frequently using fixtures.
- mobile/tests/responsive.test.mjs: responsive sizing logic, not native screen E2E.

API request conventions

Use /api/v1 as the prefix. JSON reads/writes should set Accept: application/json, writes Content-Type: application/json, authenticated calls Authorization: Bearer TOKEN, and X-CampusFlow-Client: web or mobile. Omitted platform defaults to web. Idempotency-Key is relevant to supported ticket requests; confirm the controller's behavior before generalizing it to every mutation.

```
curl -i http://localhost:8001/api/v1/health
curl -s http://localhost:8001/api/v1/auth/login \
  -H 'Accept: application/json' \
  -H 'Content-Type: application/json' \
  -H 'X-CampusFlow-Client: web' \
  --data '{"email": "student@campusflow.edu", "password": "password123"}'
```

Keep the returned bearer token private and substitute it in an API client such as Postman or a local shell variable. Do not paste tokens into defect reports.

```
curl -i http://localhost:8001/api/v1/student/timetable \
  -H 'Accept: application/json' \
```

```
-H "Authorization: Bearer $TOKEN" \  
-H 'X-CampusFlow-Client: web'
```

Read endpoint data to get real IDs. Codes are accepted on some room/detail surfaces, while other routes require UUIDs; do not globally substitute STB-101 for every {id}. Use `php backend/artisan route:list --path=api/v1` for the installed route registry. Controllers and client call signatures define required payload fields; a valid route is not necessarily a valid request body.

For a complete reset-password API test, submit all four fields: email, token, password and `password_confirmation`. Use a fresh development token or the token from configured mail delivery. The current web helper only sends token and password, which is why its route check passes but the UI cannot complete the backend contract.

Evidence and release decision

Use one row per case/action: Case ID | Build/device/role | Inputs/fixture IDs | Expected | Actual | Pass/Fail/Blocked/Not run | Evidence | Defect/owner. For CRUD cases split create, edit and delete into separate execution rows. Also list every concrete button/menu action encountered in the current build; if it is not named here, attach it to the matching domain and apply section 5.4. This catches small UI affordances added after this baseline.

Suggested acceptance gate: static checks, lint, Laravel tests on isolated PostgreSQL, browser suite and builds green; critical auth/role/ownership, ticket lifecycle and concurrency cases pass; physical-device camera/location tested; known unsupported realtime/push/reset flows resolved or explicitly excluded from release scope with owner approval. Do not sign off a feature based only on mocked browser tests.

8. Known gaps and troubleshooting

Source-observed limitations at this baseline

1. Reset password: `frontend/lib/api/endpoints.ts` and `/reset-password` submit token + password; `AuthController` requires email + `password_confirmation` as well. Predicted 422; record AUTH-08 as a known candidate failure. Use a correct API payload only to test the backend, not to claim the UI works.
2. Realtime: `frontend/lib/realtime/realtime-context.tsx` uses a custom `/api/ws` protocol. `Laravel routes/channels.php` contains only the default user channel and no matching custom socket implementation was found. `Reverb` is a dependency, but root dev starts neither `Reverb` nor a compatible gateway. No backend event/push sender was demonstrated in the reviewed code. Poll/refresh to verify data and track live delivery as unverified/incomplete.
3. Automatic timeout handling: a setting and ticket timeout fields are not proof of scheduled enforcement. `backend/routes/console.php` has only the `inspire` command; no expiration schedule was found there. Run QUE-14 and inspect actual lifecycle behavior before advertising automatic no-show processing.
4. PostGIS: health detects the extension, but `migration/model` code uses `coordinate/JSON` spatial data; enabling the extension does not establish spatial indexes or PostGIS routing.
5. Seeder: campus rows are not rerunnable idempotently, and academic/demo locations are time/location specific. Use fresh disposable QA databases and locally appropriate fixtures.
6. Test isolation: `committed phpunit.xml` defaults to the main local database and development credentials. Follow section 5.1 before any Laravel test.
7. Documentation drift: `README/mobile README` and early audit documents describe some old paths or architectural intentions. Prefer current routes, permission registry and screen code. The earlier implementation audit is historical, not this guide's runtime test result.
8. Coverage: no claim of full integration, capacity concurrency, native accessibility or push-delivery correctness is made by the static checks completed for this guide.

Troubleshooting by symptom

- composer/platform error: confirm CLI PHP version/extensions with `php -v` and `php -m`; run `composer check-platform-reqs`. Do not ignore platform requirements.
- Missing APP_KEY / encryption error: create `backend/.env` and run `key:generate` once. Regenerating a key can invalidate encrypted data; never rotate it casually on an existing environment.
- Database refused / SQLSTATE authentication error: verify database running, `DB_HOST/PORT`, login/password/database ownership, then `config:clear`. Use the mapped host port, not a container-only address.
- Duplicate seed code/constraint: seeding ran on existing data. Inspect first; only for disposable data use `db:reset`. A reset is not a migration repair for a shared environment.
- 502 API_UNAVAILABLE in browser: direct API health first, then `API_PROXY_TARGET` and port/bind configuration. `npm run dev` requires `backend/vendor` and `frontend/node_modules`.
- HTTP 500: read `backend/storage/logs/laravel.log`, inspect request ID/time and database migrations; redact secrets before sharing. Syntax checks alone cannot rule out runtime errors.
- Phone network error: use LAN IP plus `/api/v1`, test health from phone browser, check firewall/Wi-Fi isolation/VPN/WSL routing. Metro tunnel does not automatically tunnel Laravel.

- Expo SDK mismatch: use compatible Expo Go or native development build. Restart Metro with `npx expo start --clear` after changing `EXPO_PUBLIC_API_URL`.
- 403 PLATFORM_NOT_SUPPORTED: likely an intentional web/mobile separation. Use student mobile for scan/live navigation/check-in, admin web for configuration. Do not bypass roles by spoofing headers.
- Student sees empty timetable: confirm current week, term, active enrolment and timetable entries. A newly registered student is not automatically enrolled in seeded courses.
- OUTSIDE_PROXIMITY: check actual QA location, configured radius/geofence/anchor, accuracy/permission and room/office coordinates. Seed geography is San Francisco.
- Staff directory empty: inspect staff scope and office membership in admin web. Do not broaden access just to make a test display data.
- Password reset fails validation: known frontend payload mismatch; inspect AUTH-08 and section 7 rather than changing the user's password in source code.
- Live counts do not update / socket disconnect: inspect the known custom-socket integration gap; refresh to distinguish delivery failure from persistence failure.
- Push absent: Expo Go deliberately skips registration; inspect development build, physical device, permission and EAS project ID. Stored device token alone does not send push.
- Playwright cannot connect: start servers first, use correct base URLs and installed Chromium; inspect fixtures/environment flags in the selected spec.

9. Operational handover and production cautions

This is a development and acceptance guide, not a hardened deployment recipe. Do not expose seeded credentials, `APP_DEBUG=true`, PHP artisan serve or an unauthenticated database to the public Internet.

For production planning: configure a supported web server/PHP process manager, HTTPS, restricted database role/network, durable backups and restore drills, stable `APP_KEY`, secure secret management, real mail transport, log retention, rate limits, monitoring, release migrations and tested rollback procedures. Review token storage/XSS risks, CORS if separate origins are introduced, and dependency audit findings. Rotate/remove demo users and review default visitor/public exposure.

The frontend production package script is next start, while the custom proxy lives in `server.mjs`. A local production-like preview can use `npm --prefix frontend run build` followed, from `frontend/`, by `NODE_ENV=production HOSTNAME=0.0.0.0 node server.mjs`. It preserves the custom proxy, but it is not an infrastructure deployment. If using next start or a hosting provider, supply equivalent API routing; `next.config.ts` otherwise hardcodes the default Laravel target for `/api/v1` rewrites.

Queue workers, Reverb and scheduled jobs require explicit deployment design and end-to-end validation for the implemented work. Starting a worker or scheduler does not create missing jobs, broadcasts or timeout logic.

A safe handover includes: machine/software versions, installed Git SHA, sanitized environment template, database provisioning and backup owner, demo versus production account list, commands to start/stop, test results and unresolved defects, domain-specific staff assignment policy, QR printing/regeneration process and emergency access procedure.

10. Source reference and maintenance

Authoritative source locations used: root package.json and scripts/dev.mjs; backend/composer.json, .env.example, phpunit.xml, routes/api.php, routes/console.php, routes/channels.php, bootstrap/app.php; database/migrations and database/seeder/DatabaseSeeder.php; Http/Controllers, Policies and Support/Access; frontend/server.mjs, next.config.ts, lib/api/client.ts, lib/api/endpoints.ts and app; mobile/package.json, src/app/, src/lib/api.ts and src/lib/notifications.ts; backend/tests, frontend/tests/responsive and mobile/tests.

Supporting documents: README.md, mobile/README.md, docs/architecture.md, docs/api.md, docs/database.md, docs/role-platform-matrix.md, docs/user-journeys.md, docs/platform-role-audit.md and docs/implementation-audit.md. Where these differ, the executable implementation is the baseline and the discrepancy is a test risk, not an undocumented assumption.

The editable source is docs/guides/guide.md. The PDF is docs/guides/guide.pdf. Regenerate with scripts/build-guide.py using the optional documentation dependencies listed beside the guide. Update this guide whenever routes, validation, roles/platforms, setup scripts or service state machines change; record the new SHA and rerun checks. No live credentials should ever appear in either artifact.